

PROJECT AND TEAM INFORMATION

Project Title

P2P File Transfer Network

Student / Team Information

Team Name:

Team member 1 (Team Lead)

Name- Bhraguraj Vishnoi

Student ID- 24131013

Gmail- Bhraguvishnoi@gmail.com



Team member 2

Name- Madhav Goyal

Student ID- 24851009

Gmail- goyalmadhav998@gmail.com@gmail.com



Team member 2

Name- Himanshu

Student ID- 24851020

Gmail- himanshu8238@gmail.com




19/10/26

PROPOSAL DESCRIPTION

Motivation

We move files from one computer to another from time to time but we often rely on cloud storage or messaging apps or USB drives or an active internet connection. These methods may be inconvenient if the file is large, privacy is a concern, or if Internet access is not available. The aim of this project is to develop a simple peer-to-peer file sharing system where two computers can send files to each other directly over a local network. Instead of uploading the file to a third party server, the file is sent from one device to another.

Also the project aims to make the process easy and safe. The receiver sees the file sender, the file name and size. They then decide to accept or reject the file. Then the transferred file is checked using SHA-256 hashing in an effort to make sure that the received file is the same file as the original file.

State of the Art / Current solution

Our idea uses a straightforward way to work. Two computers that have the application can find each other on the local network by using UDP broadcast and then send the file through a TCP connection.

This makes the system good for cases where two computers that are close, by need to share files directly without needing cloud storage.

Our proposed solution takes an approach. Two computers running the application can discover each other on the local network using UDP broadcast and transfer the file through a TCP connection. I think this solution works well for cases where two nearby computers want to share files without using cloud storage..

Project Goals

The main goals of the project are:

1. Create a peer-, to-peer file sharing application using C#.
2. Let files move directly from one computer to another.
3. Make it work between Windows and macOS.
4. Find devices that're close by on the same local network.
5. Let the person getting the file decide to accept or turn down the request.
6. Show how much of the file has been sent.
7. Check the file after it is sent using SHA-256 hashing.
8. Store any files that are received in a folder.
9. Offer a simple.

Project Approach

The project follows a direct and practical approach.

First, both computers run the same application. Each computer starts a TCP listener on port 5000 for file transfers and a UDP service on port 5001 for discovering other devices.

When the application is running, each device periodically broadcasts its device name and IP address over UDP. Other devices listening on the same network can receive this information and add the device to their nearby-device list.

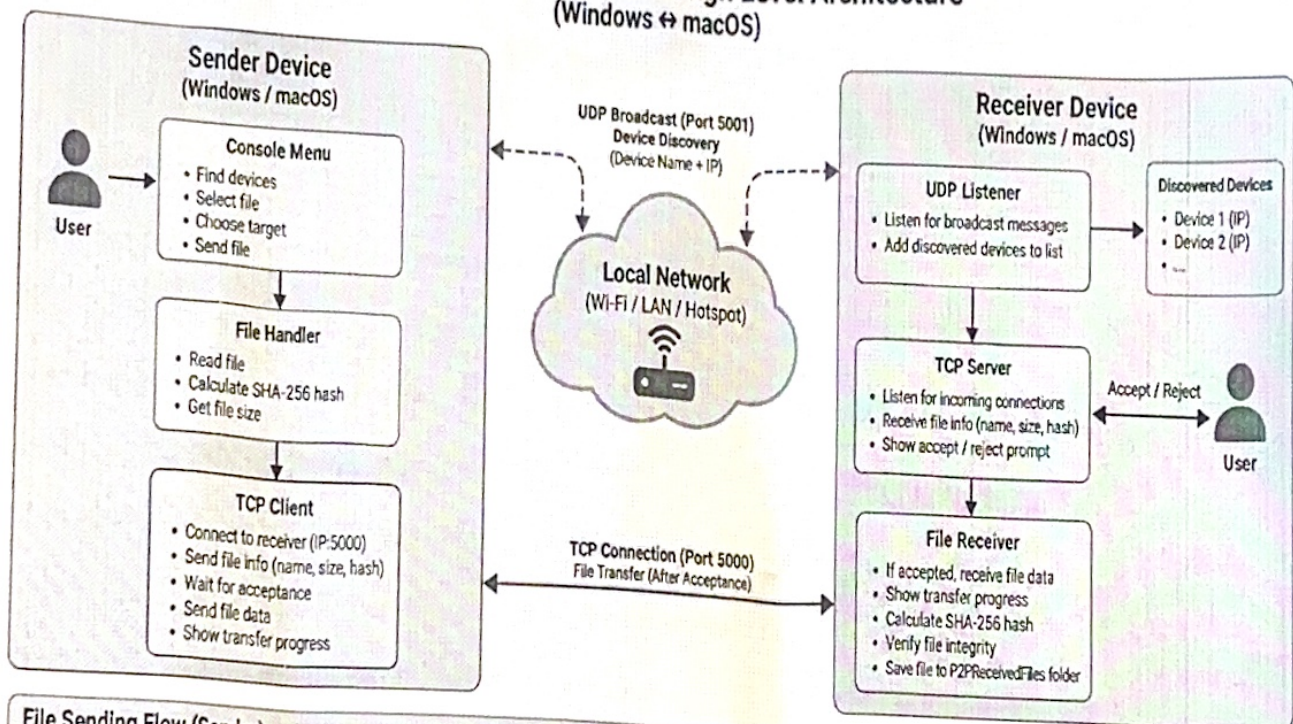
When a user wants to send a file:

1. The sender selects a file.
2. The application checks whether the file exists.
3. The file size and SHA-256 hash are calculated.
4. A TCP connection is established with the receiver.
5. File information is sent to the receiver.
6. The receiver is asked to accept or reject the file.
7. If accepted, the actual file data is transferred.
8. The receiver calculates the SHA-256 hash of the received file.
9. The calculated hash is compared with the sender's hash

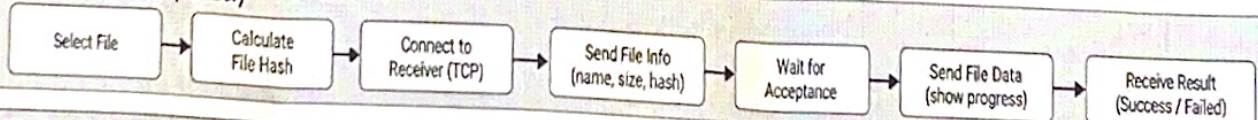
System Architecture (High Level Diagram)

The design splits the process of finding devices from sending files. UDP is used to locate devices. TCP is responsible, for moving the files in a way. The code sets up these services on UDP port 5001 and TCP port 5000..

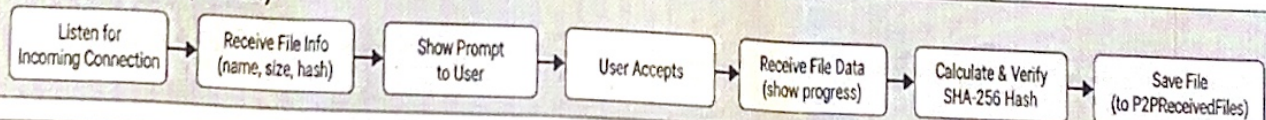
P2P File Sharing System – High Level Architecture (Windows ↔ macOS)



File Sending Flow (Sender)



File Receiving Flow (Receiver)



← - - - - - → UDP Communication (Port 5001) – Device Discovery

↔ TCP Communication (Port 5000) – File Transfer

Received files are saved in:
P2PReceivedFiles (in user's home directory)

Project Outcome / Deliverables

At the end of the project the expected outcome is a working peer-to-peer file sharing application.

The main deliverables include the following:

- A C# console-based P2P file-sharing application.
- File transfer between Windows and Windows systems.
- File transfer between Windows and macOS systems.
- File transfer between macOS and Windows systems.
- Automatic device discovery using UDP.
- File transfer using TCP.
- Manual IP-based file transfer where users can enter the IP address directly.
- File selection through the operating systems file picker.
- The receiver can. Reject incoming file transfers.
- Real-time display of file transfer progress.
- SHA-256 file integrity verification to ensure files are not corrupted.
- Automatic creation of a folder named received-files to store files.
- Connection testing using a PING/PONG mechanism.
- Troubleshooting information to help diagnose connection problems.

The application also supports a connection test. In this test the sender sends a PING message. Waits for a PONG response, from the other computer. This helps confirm that the connection is active and communication is possible.

Assumptions

The project is based on the following assumptions:

1. Both computers must have the .NET environment needed to run the C# application.
2. Both devices must be connected to the local network if automatic discovery is used
3. TCP port 5000 must be available for file transfer.
4. UDP port 5001 must be available for device discovery.
5. The operating system firewall must allow the network communication.
6. The two devices must be able to communicate with each other over the network.
7. The user must have permission to access the file being sent.
8. The receiver must have storage space for the incoming file.
9. Automatic discovery might not work on networks that block broadcast or peer-to-peer traffic.
10. The system is mainly designed for transfers between trusted devices, on the same local network.

The project's own troubleshooting section points out that same-network connectivity, firewall access, VPN interference and public or college Wi-Fi restrictions are practical considerations.

References

- Microsoft .NET Documentation – Networking and TCP/UDP communication.
- Microsoft C# Documentation – C# programming language and asynchronous programming.
- Microsoft .NET Documentation – TcpClient, TcpListener, UdpClient, and NetworkStream.
- Microsoft .NET Documentation – System.Security.Cryptography.SHA256.
- Apple macOS documentation – osascript and file-selection functionality.
- Project source code – P2P File Sharing – Windows ↔ macOS, developed in C#